Министерство науки и высшего образования Российской Федерации
Российский химико-технологический университет имени Д. И. Менделеева

Факультет цифровых технологий и химического инжиниринга
Кафедра информационных компьютерных технологий

ОТЧЕТ

ПО ЛАБОРАТОРНОМУ ПРАКТИКУМУ

«Использование численных методов в табличном процессоре Microsoft Excel»

Вариант № 66

ВЫПОЛНИЛ: Студент группы КС-24 Осолодков А.С.

ПРОВЕРИЛ: к.т.н., доцент Папаев П.Л.

Москва
2024

СОДЕРЖАНИЕ

1. Лабораторная работа 1. Численный расчёт производных. 3
1.1. Цель работы. Задача. Вариант задания 3
1.2. Теоретическая часть 3
1.3. Практическая часть 4
1.4. Выводы по работе 6
2. Лабораторная работа 2. Численные методы и алгоритмы решения нелинейных алгебраических уравнений. 7
2.1. Цель работы. Задача. Вариант задания 7
2.2. Теоретическая часть 7
2.3. Практическая часть 10
2.4 Выводы по работе 15

1. Лабораторная работа 1. Численный расчёт производных.

1.1. Цель работы. Задача. Вариант задания

Цель работы. Закрепление знаний о методе численного расчёта производных; получение навыков численного расчёта производных различных порядков и построения графиков функций с использованием табличного процессора MS Excel.

Задача. С использованием метода численного расчёта производных найти производные первого порядка для заданной функции в табличном процессоре MS Excel. Оформить результаты расчётов в табличной и графической формах. Провести исследование и сделать выводы о влиянии заданной величины приращения аргумента на ошибку расчёта производной.

Вариант задания. №16. y = 1,07·x·sin(x) +1,92·sin2 (x) –1,66·x.

1.2. Теоретическая часть

Аналитически производную функции в точке определяют как предел отношения приращения функции Δy к соответствующему приращению аргумента Δx при условии Δx→0:

$$f'(x_0) = \lim_{\Delta x \to 0} \frac{\Delta f}{\Delta x} = \lim_{\Delta x \to 0} \frac{f(x_0 + \Delta x) - f(x_0)}{\Delta x}.$$

Производная функции в точке в геометрическом смысле – это угловой коэффициент касательной к графику функции y = f(x) в этой точке.

При вычислении также можно использовать не полное приращение Δx, а половину от приращения, таким образом, что $\Delta f = f\left(x_0 + \frac{\Delta x}{2}\right) - f\left(x_0 - \frac{\Delta x}{2}\right)$. Тогда при Δx→0 формулу аналитического определения производной можно приближённо представить в виде:

$$\lim_{\Delta x \to 0} \frac{f(x_0 + \Delta x) - f(x_0)}{\Delta x} \approx \frac{f\left(x_0 + \frac{\Delta x}{2}\right) - f\left(x_0 - \frac{\Delta x}{2}\right)}{\Delta x}.$$

Полученная формула используется в численном расчёте производных. Приращение Δx, в таком случае, можно представить как Δx = 2ε, где ε – желаемая точность вычисления, заданная до выполнения алгоритма. Тогда формула для численного расчёта производной первого порядка примет вид:

$$f'(x) = \frac{f(x+\varepsilon) - f(x-\varepsilon)}{2\varepsilon}. \quad (1.1)$$

Абсолютная ошибка – разность между истинным значением величины и результатом ее изменения

1.3. Практическая часть

Исходя из поставленной задачи, требуется найти производную первого порядка для данной функции вида:

$$y = 1{,}07 \cdot x \cdot \sin(x) + 1{,}92 \cdot \sin 2(x) - 1{,}66 \cdot x$$

на интервале [–5; 5] с шагом 0,25 по независимой переменной *x*.

Для выполнения лабораторной работы используется табличный процессор MS Excel. Он позволяет автоматически обновлять значения при изменении исходных данных, что дает возможность быстро исследовать влияние различных значений приращения аргумента на точность вычислений.

На Листе 1 книги MS Excel приводятся выражения исходной функции и её первой производной, полученное аналитически:

$$y' = 3{,}84 * sin(x) * cos(x) + 1{,}07 * sin(x) + 1{,}07 * x * cos(x) - 1{,}66$$

Также на Листе 1 составляется таблица значений заданной функции в пределах изменения независимой переменной с заданным шагом, по которой необходимо построить график заданной функции.

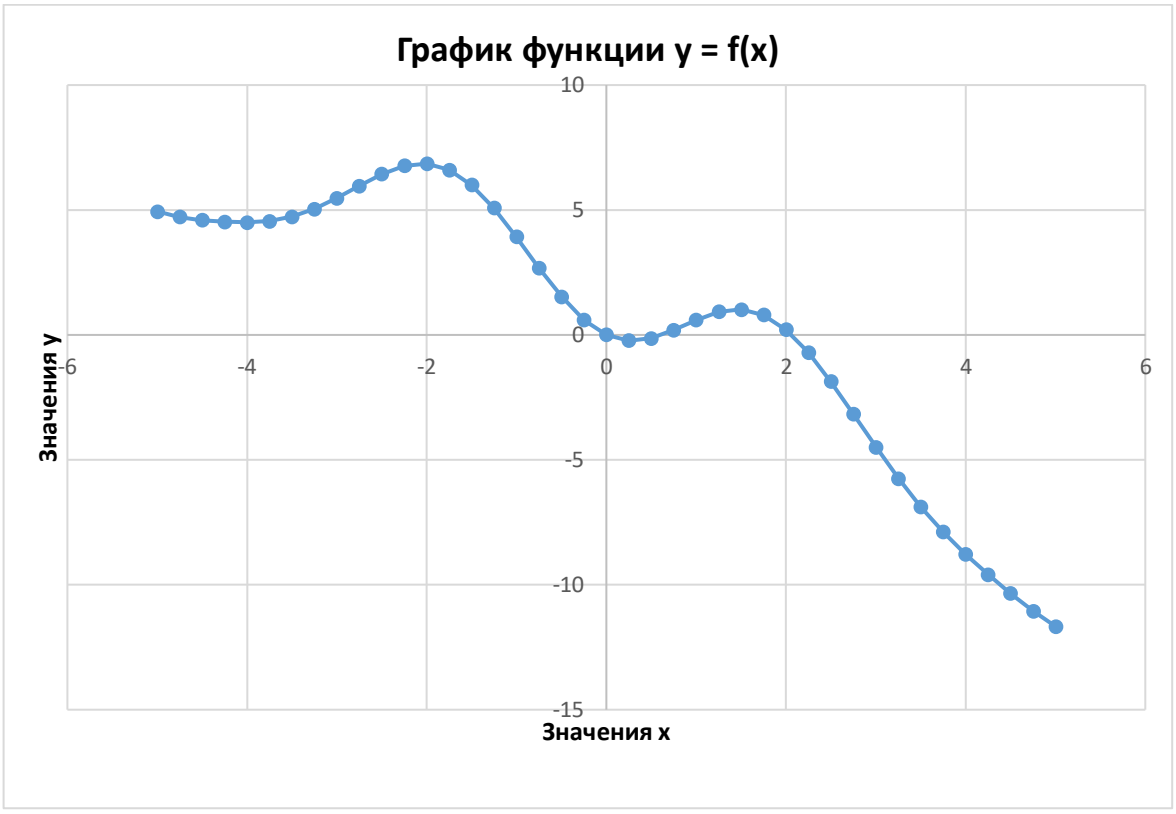


Рис. 1.1 График функции y = f(x)

На Листе 2 составляется таблица для построения графиков функциональных зависимостей исходной функции и аналитического выражения её первой производной, полученного ранее, в пределах изменения независимой переменной с заданным шагом. Оба графика строятся в единой системе координат

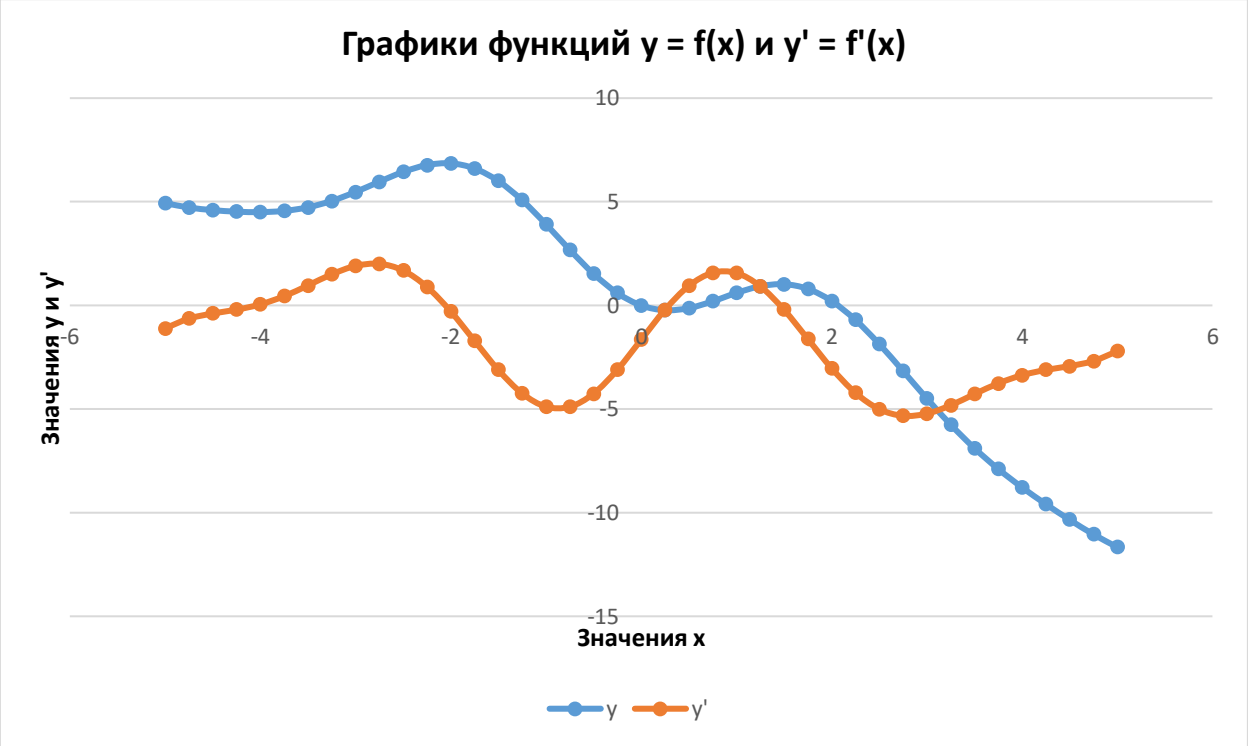


Рис. 1.2 График функции y = f(x) и y’ = f’(x)

На Листе 3 составляется таблица с результатами численного расчёта производной первого порядка в пределах изменения независимой переменной с заданным шагом. Вычисления производятся для различных значений приращения аргумента Δx: 1.0, 0.5, 0.2, 0.1; с помощью формулы численного расчёта производной первого порядка. Используя аналитическое выражение, необходимо найти абсолютные ошибки (АО) численного расчёта производных в каждой точке.

На Листе 4 строится график зависимости средней ошибки численного расчёта производной от величины приращения аргумента. Среднюю ошибку вычисляют как среднее арифметическое всех значений АО для каждого приращения Δx.

По построенному графику зависимости средней ошибки от величины приращения нужно выполнить анализ зависимости и сделать выводы о влиянии выбора приращения аргумента на величину ошибки численного расчёта производной.

1.4. Выводы по работе

В рамках этой лабораторной работы была вычислена производная первого порядка для заданной функции с помощью программы MS Excel. Результаты представлены в виде таблиц и графиков.

Анализ полученных данных показал, что изменение аргумента влияет на погрешность вычисления производной прямо пропорционально. Увеличение количества вариантов изменения независимой переменной позволяет более детально изучить эту функциональную зависимость.

Таким образом, при решении задач численного дифференцирования для более точного результата рекомендуется использовать как можно более маленькое изменение аргумента. Этот метод является прямым, поэтому увеличение точности не повлияет на скорость вычислений.

2. Лабораторная работа 2. Численные методы и алгоритмы решения нелинейных алгебраических уравнений.

2.1. Цель работы. Задача. Вариант задания

Цель работы. закрепление знаний о численных методах решения нелинейных алгебраических уравнений; получение навыков численного решения нелинейных алгебраических уравнений с использованием языка программирования Python.

Задача. С использованием одного из численных методов найти корни нелинейного алгебраического уравнения в табличном процессоре MS Excel. Оформить результаты расчетов в табличной форме. Провести исследование и сделать выводы о влиянии заданной точности на скорость решения уравнения. Сравнить между собой численные методы по скорости решения.

Вариант задания. №66 $y= -0.86 - 4.44x + 1.27x^2 + 3.25\sin(x)$

2.2. Теоретическая часть

Уравнение вида $f(x) = 0$ называется алгебраическим уравнением с одним неизвестным. Если функция $y = f(x)$ – нелинейная, то соответствующее ей уравнение называется нелинейным алгебраическим уравнением.

Всякое значение C, обращающее алгебраическое уравнение в тождество при $x = C$, называется корнем уравнения. Решить уравнение – найти все его корни или доказать, что корней нет.

В вычислительной математике процесс решения уравнения состоит из двух основных этапов:

1. Отделение корней – нахождение интервалов локализации всех корней. С одной стороны, каждому корню должен соответствовать непрерывный интервал его локализации, границы которого определяются на основе визуального (графического) или математического (численного) анализа функции $y = f(x)$. С другой стороны, на интервале обязательно должен быть единственный корень. Интервалы локализации соседних корней могут частично пересекаться.

2. Уточнение корней – определение корней уравнения на каждом интервале локализации с заданной точностью с использованием одного из численных методов.

Наиболее часто задача отделения корней решается с использованием графических методов, позволяющих приближенно найти корень уравнения или интервал его локализации по внешнему виду графика функции y = f(x).

В данной работе используется метод точек пересечения графиков функций предусматривает преобразование уравнения f(x) = 0 к виду $f_1(x) = f_2(x)$. Результатом приближенного определения корней данным методом являются точки пересечения графиков функций $y = f_1(x)$ и $y = f_2(x)$.

Решение нелинейного алгебраического уравнения методом простых итераций требует преобразования исходного уравнения f(x) = 0 к виду x =φ(x).

Достаточным условием сходимости метода является φ'(x) <1 , выполняющееся на всем интервале локализации корня. Причем чем меньше по величине абсолютное значение производной, тем быстрее будет получено решение.

Последовательность приближений строится с использованием итерационного соотношения:

$$x^{(k)} = \varphi\left(x^{(k-1)}\right), \tag{3.3}$$

до тех пор, пока не будет выполнено условие окончания:

$$\left|x^{(k)} - x^{(k-1)}\right| \le \varepsilon. \tag{3.4}$$

Во многих случаях представляется сложным получить итерационную форму уравнения (3.3), тогда можно воспользоваться другой формой:

$$x^{(k)} = x^{(k-1)} + \lambda \cdot f\left(x^{(k-1)}\right),$$

получающейся из уравнения вида λ · f (x) = 0 , равносильного исходному. При выборе параметра λ необходимо контролировать выполнение достаточного условия сходимости метода простых итераций. В качестве формулы для его расчета можно использовать следующую:

$$\lambda = -\frac{\operatorname{sgn}\left(f'\left(x^{(0)}\right)\right) \cdot \nu}{1 + \left|f'\left(x^{(0)}\right)\right|},$$

где ν – коэффициент, влияющий на скорость сходимости метода, выбираемый,

как правило, в пределах от 0,3 до 0,7; sgn( ) – функция знака, возвращающая значения: 1, если аргумент – положительное число, или –1, если аргумент – отрицательное число. Производная в начальной точке не должна равняться нулю, а расчитывается по формуле для численного расчёта производной (1.1).

В данной лабораторной работе используется интервальный метод половинного деления. Алгоритм решения методом половинного деления следующий:

1. Задаются точность поиска ε и начальный интервал локализации корня [a, b]; определяются значения функции y = f(x) на его концах: f(a), f(b)

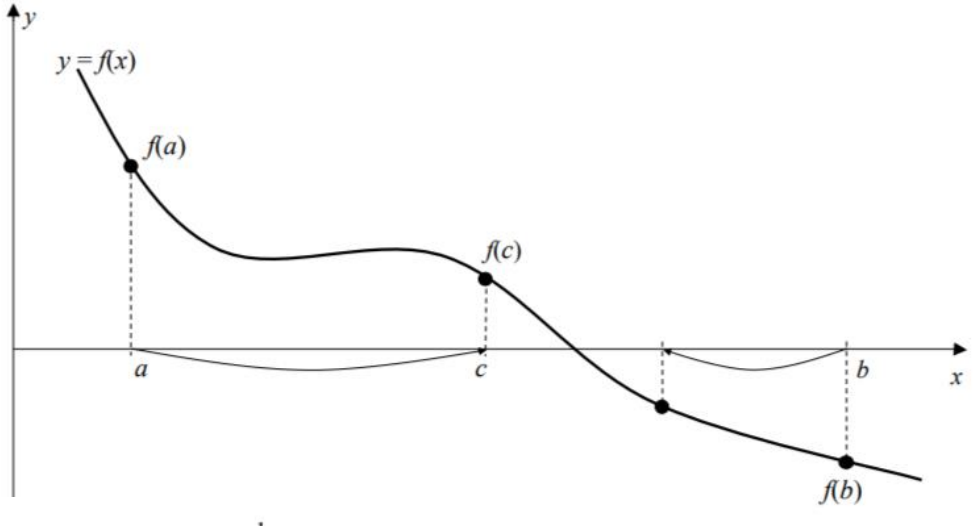


Рис. 2.2.1. Графическая интерпретация метода половинного деления

2. Текущий интервал локализации [a, b] делится пополам точкой c = (a +b) 2 и находится значение f(c);

3. Проверяются условия:

$$(b - a)/2 \le \varepsilon; \quad (3.1)$$

$$f(c) = 0. \quad (3.2)$$

Если хотя бы одно из условий (2.1) выполняется, вычислительный процесс останавливается, а число c является корнем уравнения на заданном интервале;

4. Проверяется условие: f (a)· f (c) > 0 . Если оно выполняется, левая граница интервала локализации переносится в точку c (a = c), в противном случае в точку c переносится правая граница (b = c). Интервал локализации на данном этапе алгоритма уменьшается вдвое;

5. Процедура повторяется с п. 2.

2.3. Практическая часть

Для решения поставленной задачи необходимо определить количество корней на графике при помощи графического метода пересечения графиков функций.

Для этого разделим исходное уравнение на дву функции:

$$f1(x)= -0.86 - 4.44x + 1.27x^2$$
$$f2(x) = -3.25\sin(x)$$

Теперь, имя две отдельные функции, необходимо найти точки пересечения их графиков. Используя библиотеку Matplotlib языка Python построим графики этих функций в одной координатной плоскости:

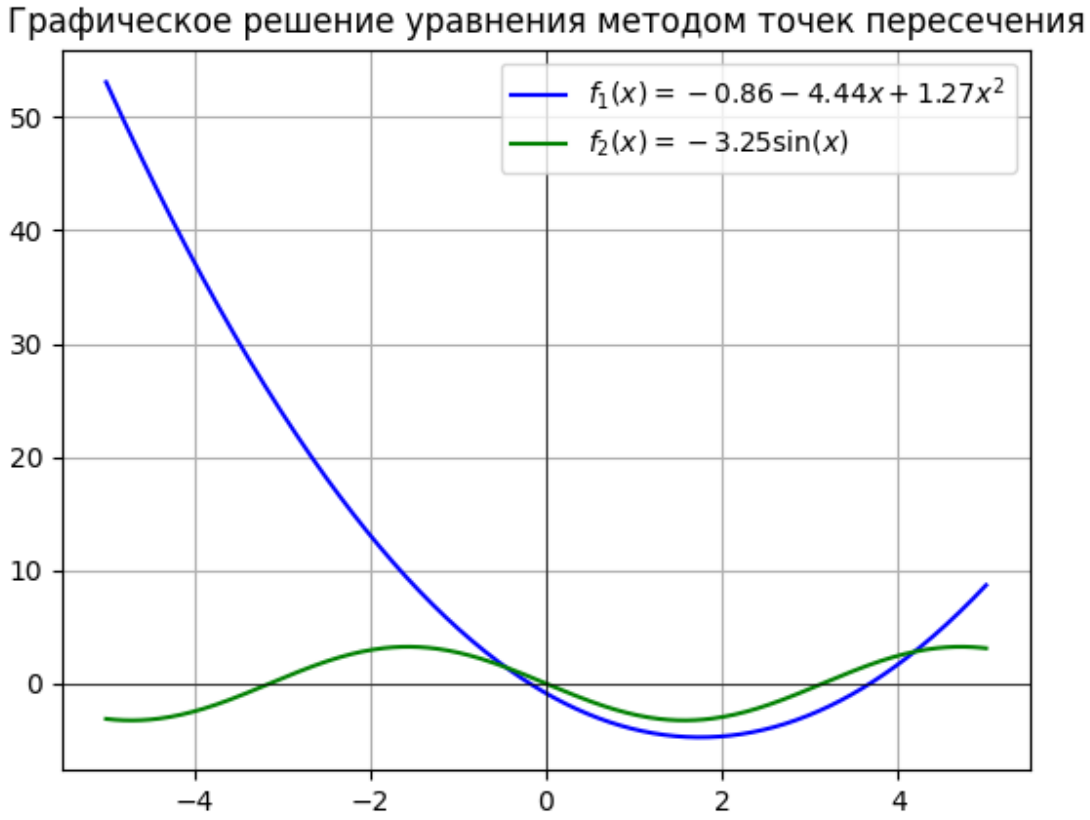


Рис. 2.3.1 Графическое решение уравнения методом точек пересечения графиков функций

На Рис. 2.3.2 продемонстрирована реализация графика на языке Python

```python
import numpy as np
import matplotlib.pyplot as plt

2 usages
def create_plot1(f11, f22):
    # Построение графика
    x_vals = np.linspace(-5,  stop: 5,  num: 400)

    # Вычисляем значения для обеих функций
    f1_vals = f11(x_vals)
    f2_vals = f22(x_vals)

    # Построение графиков
    plt.plot( *args: x_vals, f1_vals, label=r'$f_1(x) = -0.86 - 4.44x + 1.27x^2$', color='blue')
    plt.plot( *args: x_vals, f2_vals, label=r'$f_2(x) = -3.25\sin(x)$', color='green')

    # Добавляем оси и сетку
    plt.axhline( y: 0, color='black', linewidth=0.5)
    plt.axvline( x: 0, color='black', linewidth=0.5)
    plt.title('Графическое решение уравнения методом точек пересечения')
    plt.legend()
    plt.grid(True)

    # Показать график
    plt.savefig('graph.png')
    plt.show()
```

Рис. 2.3.2 Реализация графика на языке Python

Далее эмпирически оцениваем интервал локализации каждого корня, а также их приблизительные значения. Вводим эти значения в программу.

Можно заметить, что первый корень находится в диапазоне [-1;0], а второй в [4, 5]. Приблизив график, выберем приблизительные значения корней -0.5 и 4. После ввода данных программа выполняет остальную работу.

На Рис. 2.3.3 и Рис. 2.3.4 приведена реализация метода простых итераций и метода половинного деления соответственно. Для метода простых итераций выберем значения из их допустимого диапазона: 0,7 и 1.

```python
# Метод простых итераций
2 usages
def simple_iteration(accuracy, x0):
    x = x0
    iterations = 0
    v, gamma = 0.7, 1
    lmb = -1 * ((sgn(x) * v) / (gamma + abs(df(x))))
    while True:
        x_next = x + lmb * f(x)
        iterations += 1
        print(f"{x:.5f}  {x_next:.5f}  {abs(x_next - x):.5f}", file=file)
        if abs(x_next - x) < accuracy:
            break
        x = x_next
    return x_next, iterations
```

Рис. 2.3.3 Реализация метода простых итераций на языке Python

```python
# Метод половинного деления
1 usage
def bisection_method(f, interval, accuracy):
    a, b = interval
    iterations = 0
    while abs(b - a) > accuracy:
        c = (a + b) / 2
        print(f"{a:.5f}  {b:.5f}  {c:.5f}  {f(a):.5f}  {f(c):.5f}  {f(a) * f(c):.5f}  {(b - a) / 2:.5f}", file=file1)
        if f(a) * f(c) < 0:
            b = c
        else:
            a = c
        iterations += 1
    return (a + b) / 2, iterations
```

Рис. 2.3.4 Реализация метода половинного деления на языке Python

Создается файл avm.txt, в него выводятся данные нахождения корня методом простых итераций при разных значениях точности, часть файла показана на Рис. 2.3.5

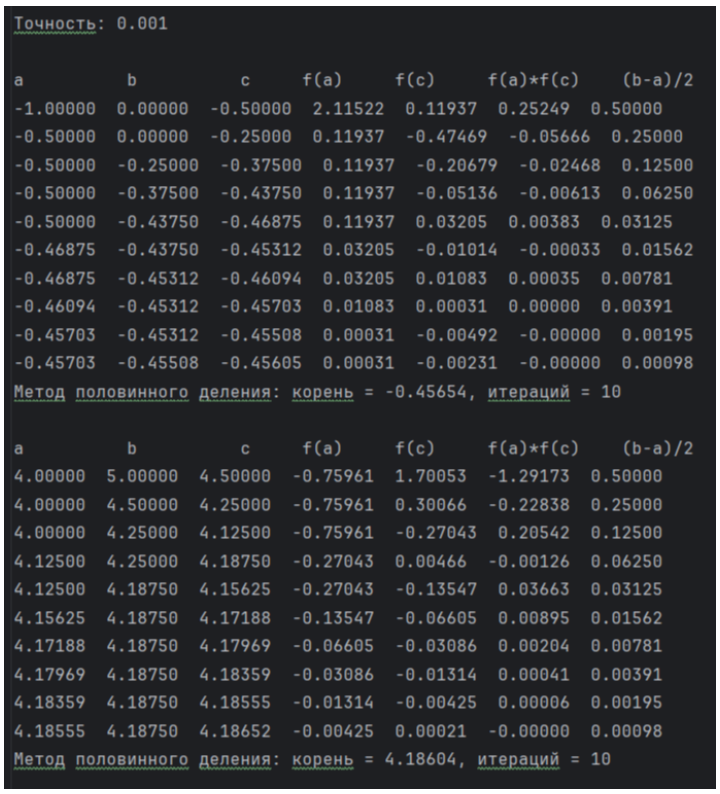
```
Точность: 1e-05

x(k-1)     x(k)     |x(k)-x(k-1)|
-0.50000  -0.47834  0.02166
-0.47834  -0.46774  0.01060
-0.46774  -0.46243  0.00531
-0.46243  -0.45973  0.00269
-0.45973  -0.45836  0.00137
-0.45836  -0.45765  0.00070
-0.45765  -0.45729  0.00036
-0.45729  -0.45711  0.00018
-0.45711  -0.45701  0.00009
-0.45701  -0.45697  0.00005
-0.45697  -0.45694  0.00002
-0.45694  -0.45693  0.00001
-0.45693  -0.45692  0.00001
Метод простых итераций: корень = -0.45692, итераций = 13

x(k-1)     x(k)     |x(k)-x(k-1)|
4.00000  4.11570  0.11570
4.11570  4.16286  0.04716
4.16286  4.17904  0.01619
4.17904  4.18419  0.00514
4.18419  4.18578  0.00159
4.18578  4.18626  0.00049
4.18626  4.18641  0.00015
4.18641  4.18646  0.00005
4.18646  4.18647  0.00001
4.18647  4.18648  0.00000
Метод простых итераций: корень = 4.18648, итераций = 10
```

Рис. 2.3.5 файл avm.txt

Создается файл avm1.txt, в него выводятся данные нахождения корня методом половинного деления при разных значениях точности, часть файла показана на Рис. 2.3.6



```
Точность: 0.001

a          b          c      f(a)      f(c)      f(a)*f(c)    (b-a)/2
-1.00000  0.00000  -0.50000  2.11522  0.11937  0.25249  0.50000
-0.50000  0.00000  -0.25000  0.11937  -0.47469  -0.05666  0.25000
-0.50000  -0.25000  -0.37500  0.11937  -0.20679  -0.02468  0.12500
-0.50000  -0.37500  -0.43750  0.11937  -0.05136  -0.00613  0.06250
-0.50000  -0.43750  -0.46875  0.11937  0.03205  0.00383  0.03125
-0.46875  -0.43750  -0.45312  0.03205  -0.01014  -0.00033  0.01562
-0.46875  -0.45312  -0.46094  0.03205  0.01083  0.00035  0.00781
-0.46094  -0.45312  -0.45703  0.01083  0.00031  0.00000  0.00391
-0.45703  -0.45312  -0.45508  0.00031  -0.00492  -0.00000  0.00195
-0.45703  -0.45508  -0.45605  0.00031  -0.00231  -0.00000  0.00098
Метод половинного деления: корень = -0.45654, итераций = 10

a          b          c      f(a)      f(c)      f(a)*f(c)    (b-a)/2
4.00000  5.00000  4.50000  -0.75961  1.70053  -1.29173  0.50000
4.00000  4.50000  4.25000  -0.75961  0.30066  -0.22838  0.25000
4.00000  4.25000  4.12500  -0.75961  -0.27043  0.20542  0.12500
4.12500  4.25000  4.18750  -0.27043  0.00466  -0.00126  0.06250
4.12500  4.18750  4.15625  -0.27043  -0.13547  0.03663  0.03125
4.15625  4.18750  4.17188  -0.13547  -0.06605  0.00895  0.01562
4.17188  4.18750  4.17969  -0.06605  -0.03086  0.00204  0.00781
4.17969  4.18750  4.18359  -0.03086  -0.01314  0.00041  0.00391
4.18359  4.18750  4.18555  -0.01314  -0.00425  0.00006  0.00195
4.18555  4.18750  4.18652  -0.00425  0.00021  -0.00000  0.00098
Метод половинного деления: корень = 4.18604, итераций = 10
```

Рис. 2.3.6 файл avm1.txt

Создается файл avm2.txt, в него выводятся данные о количестве выполненных итераций для нахождения разных корней обоими методами в отдельности. Часть файла показана на Рис. 2.3.7

Точность (ε)	Log(ε)	Метод половинного деления (x1)	Метод простых итераций (x1)	Метод половинного деления (x2)	Метод простых итераций (✓13 ^
0.10000	-1.0	4	1	4	2
0.01000	-2.0	7	3	7	4
0.00100	-3.0	10	6	10	6
0.00010	-4.0	14	9	14	8
0.00001	-5.0	17	13	17	10

Рис. 2.3.7 файл avm2.txt

На экран выводится график зависимости количества выполненных итераций от значения точности в логарифмической шкале отдельно для каждого корня и метода.

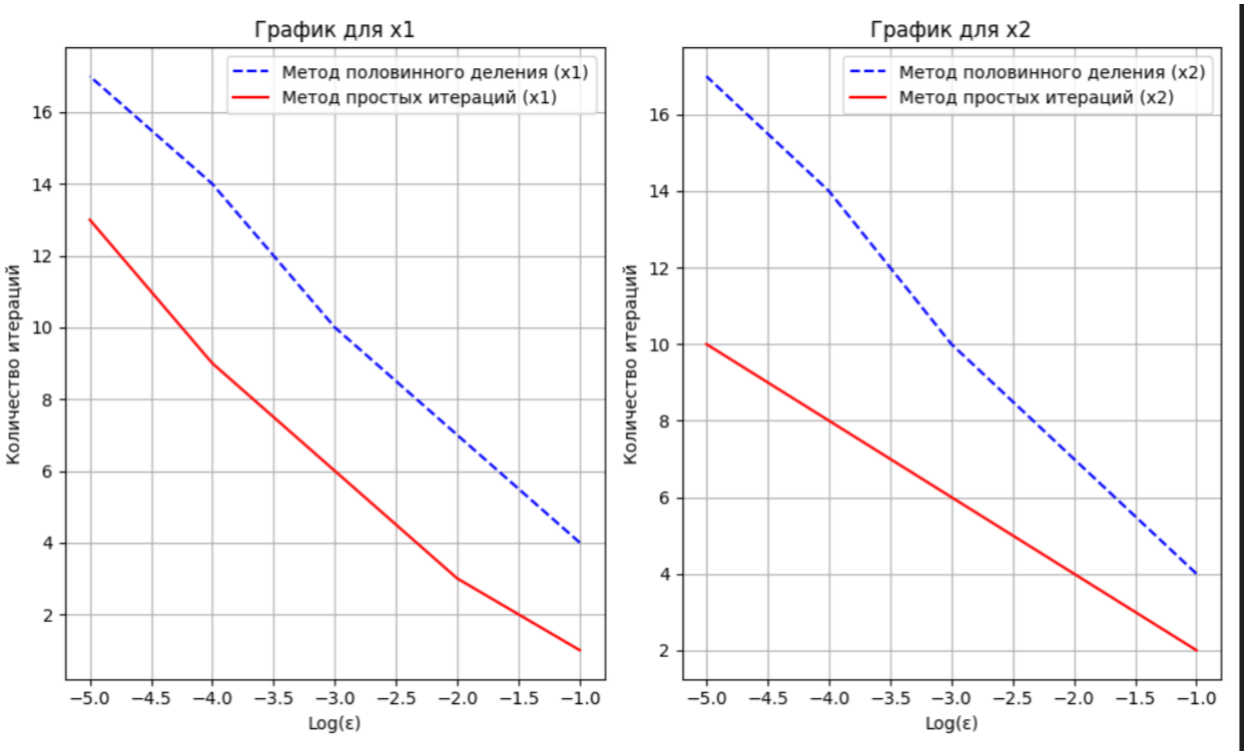


Рис. 2.3.8 График зависимости количества итераций от точности

График на Рис. 2.3.8 был построен с помощью библиотеки matplotlib языка Python. Его реализация приведена на Рис. 2.3.9

```python
def create_plot2(log_eps, bis_iter, bis_iter2, sim_iter, sim_iter2):
    # Построение графиков
    plt.figure(figsize=(10, 6))

    # График 1: для x1
    plt.subplot(*args: 1, 2, 1)
    plt.plot(*args: log_eps, bis_iter, 'b--', label="Метод половинного деления (x1)")
    plt.plot(*args: log_eps, sim_iter, 'r-', label="Метод простых итераций (x1)")
    plt.xlabel('Log(ε)')
    plt.ylabel('Количество итераций')
    plt.title('График для x1')
    plt.legend()
    plt.grid(True)

    # График 2: для x2
    plt.subplot(*args: 1, 2, 2)
    plt.plot(*args: log_eps, bis_iter2, 'b--', label="Метод половинного деления (x2)")
    plt.plot(*args: log_eps, sim_iter2, 'r-', label="Метод простых итераций (x2)")
    plt.xlabel('Log(ε)')
    plt.ylabel('Количество итераций')
    plt.title('График для x2')
    plt.legend()
    plt.grid(True)

    # Показать графики
    plt.tight_layout()
    plt.savefig('x2.png')
    plt.show()
```

Рис. 2.3.9 Реализация графика на языке Python

2.4 Выводы по работе

В обоих методах наблюдается закономерность: с повышением точности увеличивается количество выполненных итераций. Графики демонстрируют схожую динамику роста. Для обоих корней метод простых итераций требует меньшего числа итераций по сравнению с методом половинного деления. Эта разница может быть обусловлена выбранными интервалами локализации и приближёнными значениями, определёнными эмпирически. Хотя разница в количестве итераций незначительна, метод простых итераций продемонстрировал более высокую эффективность.

3. Лабораторная работа 3. Численное решение систем линейных алгебраических уравнений.

3.1. Цель работы. Задача. Вариант задания

Цель работы. закрепление знаний о численных методах решения систем линейных алгебраических уравнений; получение навыков численного решения систем линейных алгебраических уравнений с использованием языка программирования Си.

Задача. С использованием одного из численных методов найти корни системы линейных алгебраических уравнений в табличном процессоре MS Excel. Оформить результаты расчетов в табличной форме. Провести исследование и сделать выводы о влиянии заданной точности на скорость решения уравнения. Сравнить между собой численные методы по скорости решения.

Вариант задания.

Номер варианта: 66

Коэффициенты:					Св. чл.:
8.8	8.8	-9.5	19.9	3.6	69.82
-3.4	5.0	14.6	-8.7	-2.2	54.40
6.7	18.6	-12.8	-5.0	-8.3	103.53
14.3	7.7	1.5	-0.4	-0.7	109.24
5.2	4.5	0.5	5.9	9.5	21.82

Начальное приближение:

1.5

0.4

-2.0

-2.0

1.6

Метод для п.4: Жордана-Гаусса

Метод для п.5: Гаусса-Зейделя с постоянным параметром лямбда

3.2. Теоретическая часть

Выполнение данной лабораторной работы требует решения системы линейных алгебраических уравнений в матричной форме. В матричной записи система из n линейных уравнений может быть представлена следующим образом:

$$\bar{\bar{A}} \cdot \vec{x} = \vec{b},$$

где A – матрица коэффициентов системы размера n × n; b – вектор свободных членов размера n; x – вектор независимых переменных размера n.

Методы решения таких систем можно разделить на две группы: прямые (точные) и итерационные. Прямые методы позволяют получить точное решение за конечное число вычислений. Итерационные методы дают бесконечный ряд последовательных приближений к решению, а вычислительный процесс останавливается в соответствии с ограничениями на точность, наложенными пользователем.

Данный вариант лабораторной работы предусматривает реализацию метода Жордана-Гаусса, относящийся к классу прямых методов, и метода простых итераций с применением модификации Гаусса-Зейделя с параметром лямбда, являющийся итерационным методом. Рассмотрим эти методы далее.

Метод Жордана–Гаусса позволяет решить систему с использованием следующего алгоритма:

1. Составляют расширенную прямоугольную матрицу за счет добавления к исходной матрице коэффициентов столбца свободных членов:

$$\bar{\bar{A}}^{\mathrm{P}} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} & b_1 \\ a_{21} & a_{22} & \dots & a_{2n} & b_2 \\ \dots & \dots & \dots & \dots & \dots \\ a_{n1} & a_{n2} & \dots & a_{nn} & b_n \end{bmatrix};$$

2. С использованием элементарных алгебраических преобразований (умножение матрицы на константу, сложение строк) расширенную матрицу приводят к виду:

$$\bar{\bar{A}}^{\mathrm{P}} = \begin{bmatrix} 1 & 0 & \dots & 0 & x_1 \\ 0 & 1 & \dots & 0 & x_2 \\ \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & \dots & 1 & x_n \end{bmatrix},$$

вследствие чего ее последний столбец представляет собой вектор решения системы. Решение системы линейных алгебраических уравнений

итерационными методами, в отличие от вышеприведенных, требует оценки точности полученного на каждой следующей итерации приближения к решению. В выполняемой работе данная оценка осуществляется с использованием евклидовой нормы. Норма – величина, которая характеризует удаленность данного вектора (матрицы) от нулевого вектора (матрицы) такого же размера. Евклидова норма некоторой произвольно взятой матрицы A размера m×n определяется по соотношению:

$$\left\|\overline{\overline{A}}\right\| = \sqrt{\sum_{i=1}^{m}\sum_{j=1}^{n} a_{ij}^{\,2}},$$

Метод простых итераций заключается в преобразовании исходной системы (4.1) к итерационной форме:

$$\vec{x} = \overline{\overline{C}} \cdot \vec{x} + \vec{d},$$

где в матрице C и векторе d:

$$c_{ij} = \begin{cases} 0, i = j; \\ -\dfrac{a_{ij}}{a_{ii}}, i \neq j; \end{cases} \quad d_i = \frac{b_i}{a_{ii}}, \quad \forall i = \overline{1, n}; j = \overline{1, n},$$

и нахождении последовательности приближений векторов независимой переменной к решению на каждом шаге по аналогичным значениям, полученным на предыдущем шаге итераций:

$$\begin{cases} x_1^{(k)} = c_{11}x_1^{(k-1)} + c_{12}x_2^{(k-1)} + \ldots + c_{1n}x_n^{(k-1)} + d_1; \\ x_2^{(k)} = c_{21}x_1^{(k-1)} + c_{22}x_2^{(k-1)} + \ldots + c_{2n}x_n^{(k-1)} + d_2; \\ \ldots \\ x_n^{(k)} = c_{n1}x_1^{(k-1)} + c_{n2}x_2^{(k-1)} + \ldots + c_{nn}x_n^{(k-1)} + d_n. \end{cases}$$

Достаточным условием сходимости данного метода является выполнение следующего условия:

$$\left\|\overline{\overline{C}}\right\| \leq 1$$

Метод простых итераций с применением модификации Гаусса-Зейделя с постоянным коэффициентом лямбда заключается в преобразовании исходной системы к итерационной форме вида:

$$\vec{x}^{(k)} = \bar{\bar{C}} \cdot \vec{x}^{(k-1)} + \vec{d},$$

где $c_{ij} = \begin{cases} 1 + \lambda_i \cdot a_{ij}, & i = j; \\ \lambda_i \cdot a_{ij}, & i \neq j; \end{cases}$ (3) $d_i = -\lambda_i \cdot b_i$; (4) $i = \overline{1,n}; j = \overline{1,m}.$

В свою очередь элементы вектора $\vec{\lambda}$ рассчитываются по формуле:

$$\lambda_i = -\frac{sgn(a_{ii}) \cdot v}{1 + |a_{ii}|},$$

где v – коэффициент, влияющий на скорость сходимости метода, выбираемый, как правило, в пределах от 0,3 до 0,7; sgn( ) – функция знака, возвращающая значения: 1, если аргумент – положительное число, или –1, если аргумент – отрицательное число.

С целью выполнения данного условия необходимо предварительно преобразовать (переставить местами, умножить на константу, сложить) уравнения исходной системы таким образом, чтобы на главной диагонали матрицы коэффициентов находились максимальные по абсолютному значению величины.

Фактически же при решении системы методом простых итераций необходимо, чтобы на каждом шаге итерации k соблюдалось условие:

$$\left\|\vec{x}^{(k)} - \vec{x}^{(k-1)}\right\| < \left\|\vec{x}^{(k-1)} - \vec{x}^{(k-2)}\right\|$$

Цикл итераций повторяют до выполнения условий окончания:

$$\begin{cases} \left\|\vec{x}^{(k)} - \vec{x}^{(k-1)}\right\| \leq \varepsilon; \\ \left\|\bar{\bar{A}} \cdot \vec{x}^{(k)} - \vec{b}\right\| \leq \varepsilon, \end{cases}$$

где ε – заданная точность решения системы.

Сходимость решения не зависит от выбранного начального приближения, однако, чем точнее начальное приближение, тем быстрее будет получено решение.

3.3. Практическая часть

Для реализации методов был использован язык Си. Перед работой с СЛАУ нужно преобразовать исходную матрицу так, чтобы выполнялось условие диагонального преобладания. После преобразование матрицы нужно найти ее итерационную форму. Исходная матрица с векторов свободных членов, матрица с преобразованием по условию диагонального преобладания и матрица в итерационной форме с вектором d выведены в файл output.txt:

output.txt

```
Матрица A:
8.80    8.80    -9.50   19.90   3.60    | 69.82
-3.40   5.00    14.60   -8.70   -2.20   | 54.40
6.70    18.60   -12.80  -5.00   -8.30   | 103.53
14.30   7.70    1.50    -0.40   -0.70   | 109.24
5.20    4.50    0.50    5.90    9.50    | 21.82

Матрица Апр:
14.30   7.70    1.50    -0.40   -0.70   | 109.24
3.30    23.60   1.80    -13.70  -10.50  | 103.53
-3.40   5.00    14.60   -8.70   -2.20   | 54.40
8.80    8.80    -9.50   19.90   3.60    | 69.82
5.20    4.50    0.50    5.90    9.50    | 21.82

Матрица C:
0.72    -0.15   -0.03   0.01    0.01    | 2.14
-0.04   0.71    -0.02   0.17    0.13    | 1.26
0.07    -0.10   0.72    0.17    0.04    | 1.05
-0.13   -0.13   0.14    0.71    -0.05   | 1.00
-0.15   -0.13   -0.01   -0.17   0.73    | 0.62
```

Рис 3.1 Преобразование матрицы

Далее преобразованная матрица коэффициентов объединяется с вектором свободных членов для решения системы методом Жордана-Гаусса:

```
Матрица A:
1.00    0.00    0.00    0.00    0.00    | 5.32
0.00    1.00    0.00    0.00    0.00    | 3.11
0.00    0.00    1.00    0.00    0.00    | 5.02
0.00    0.00    0.00    1.00    0.00    | 2.93
0.00    0.00    0.00    0.00    1.00    | -4.17


Решение методом Жордана-Гаусса:
5.32    3.11    5.02    2.93    -4.17
```

Рис 3.2 Решение СЛАУ методом Жордана-Гаусса

Итерационная форма уже найдена и записана в матрицу C. Используем ее для решения СЛАУ методом Гаусса-Зейделя с постоянным коэффициентом лямбда при заданных значениях точности: 0.1, 0.02, 0.005, 0.001:

```
Решение для точности 0.100:
x[0] = 5.459    x[1] = 3.033    x[2] = 4.609    x[3] = 2.360    x[4] = -3.583
Итераций: 16

Решение для точности 0.020:
x[0] = 5.358    x[1] = 3.083    x[2] = 4.948    x[3] = 2.824    x[4] = -4.061
Итераций: 27

Решение для точности 0.005:
x[0] = 5.329    x[1] = 3.100    x[2] = 5.003    x[3] = 2.907    x[4] = -4.145
Итераций: 36

Решение для точности 0.001:
x[0] = 5.320    x[1] = 3.106    x[2] = 5.017    x[3] = 2.929    x[4] = -4.167
Итераций: 46
```

Рис 3.3 Решение СЛАУ методом Гаусса-Зейделя

Далее построим столбчатую диаграмму, которая будет отражать зависимость количества итераций от выбранного значения точности. Реализуем диаграмму при помощи приложения gnuplot внутри основной программы:

```c
// Создаем файл с командами для Gnuplot
FILE *gnuplot_script = fopen("gnuplot_commands.gp", "w");
if (gnuplot_script == NULL) {
    printf("Ошибка при открытии файла для записи команд Gnuplot.\n");
    return 1;
}

// Записываем команды для построения столбчатой диаграммы
fprintf(gnuplot_script, "set terminal pngcairo size 800,600 enhanced font 'Verdana,12'\n");  // Устанавливаем вывод в PNG
fprintf(gnuplot_script, "set output 'bar_chart.png'\n");  // Имя выходного файла
fprintf(gnuplot_script, "set boxwidth 0.5\n");  // Ширина столбцов
fprintf(gnuplot_script, "set style fill solid\n");  // Заливка столбцов
fprintf(gnuplot_script, "set xlabel 'Точность'\n");  // Метка оси X
fprintf(gnuplot_script, "set ylabel 'Количество итераций'\n");  // Метка оси Y
fprintf(gnuplot_script, "set title 'Количество итераций, необходимых для достижения заданной точности'\n");  // Заголовок
fprintf(gnuplot_script, "set grid ytics\n");  // Сетка по оси Y

// Устанавливаем равномерные метки на оси X и подписываем их
fprintf(gnuplot_script, "set xtics ('0.1' 1, '0.02' 2, '0.005' 3, '0.001' 4)\n");

// Построение столбцов
fprintf(gnuplot_script, "plot 'data.dat' using 1:2 with boxes lc rgb 'gray' title ''\n");

fclose(gnuplot_script);

// Запуск Gnuplot для выполнения команд
system("gnuplot gnuplot_commands.gp");

printf("График создан и сохранен в 'bar_chart.png'.\n");
```

Рис 3.4 Построение стобчатой диаграммы на языке Си

Полученная диаграмма выглядит так:

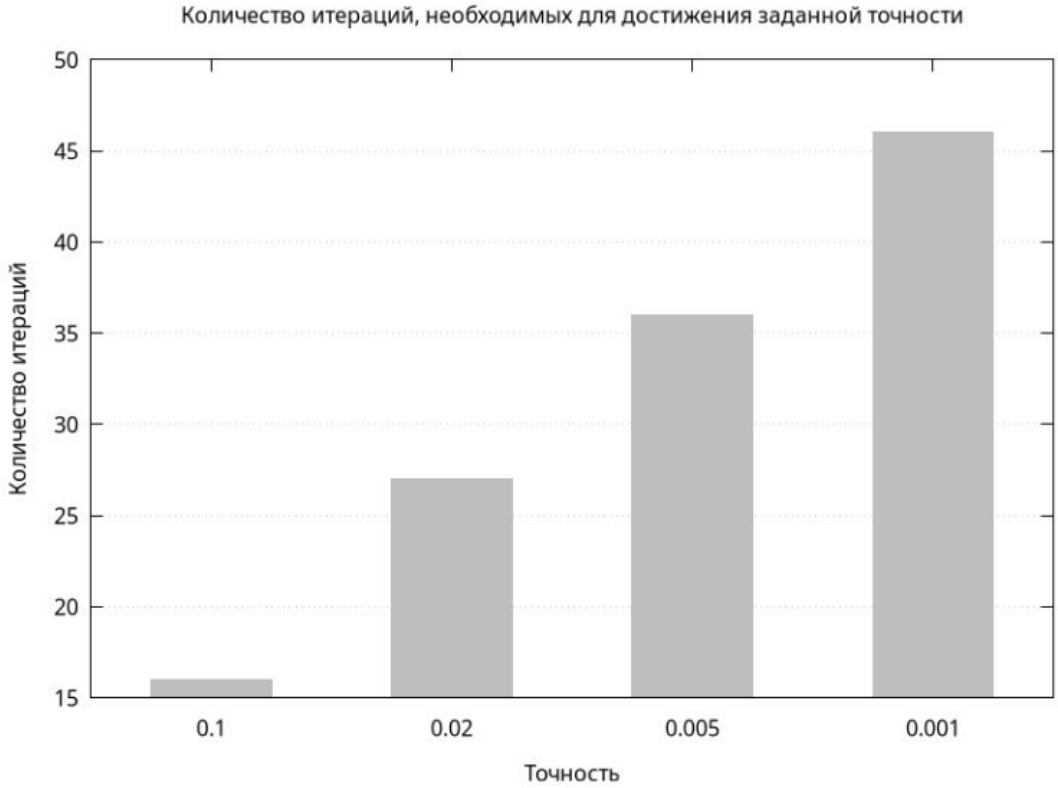


Рис 3.5 Диаграмма зависимости количества итераций от точности

Далее нужно построить график изменения элементов вектора X по мере увеличения количества итераций для выбранного значения точности 0.1. Для начала создадим матрицу, число строк которой будет соответствовать количеству итераций, а количество столбцов элементам вектора X. Получаем матрицу 16x5:

x0	x1	x2	x3	x4
3.23	1.33	-0.58	-1.16	1.34
4.29	2.03	0.58	-0.62	0.80
4.91	2.50	1.47	-0.21	0.17
5.26	2.78	2.15	0.12	-0.44
5.44	2.94	2.67	0.42	-0.99
5.52	3.02	3.06	0.69	-1.47
5.55	3.05	3.37	0.94	-1.87
5.56	3.05	3.62	1.18	-2.21
5.55	3.05	3.82	1.39	-2.49
5.54	3.04	3.99	1.59	-2.73
5.53	3.03	4.14	1.76	-2.93
5.51	3.03	4.26	1.91	-3.11
5.50	3.03	4.37	2.05	-3.25
5.49	3.03	4.46	2.17	-3.38
5.47	3.03	4.54	2.27	-3.49
5.46	3.03	4.61	2.36	-3.58

Рис 3.6 Матрица значений X при точности 0.1

Вновь воспользуемся gnuplot для построения графика:

```c
FILE *file = fopen("data2.txt", "w");
if (file == NULL) {
    perror("Не удалось открыть файл для записи");
    return 1;
}

// Записываем матрицу в файл
for (int i = 0; i < ROWS; i++) {
    for (int j = 0; j < SIZE; j++) {
        fprintf(file, "%.2lf ", data1[i][j]);
    }
    fprintf(file, "\n");
}

fclose(file);  // Закрываем файл

// Теперь генерируем скрипт для gnuplot
FILE *gnuplot = popen("gnuplot -persistent", "w");
if (gnuplot == NULL) {
    perror("Не удалось открыть gnuplot");
    return 1;
}

fprintf(gnuplot, "set terminal png size 800,600\n");
fprintf(gnuplot, "set output 'output.png'\n");
// Пишем команды для GNUPlot
fprintf(gnuplot, "set title 'График изменения всех элементов вектора'\n");
fprintf(gnuplot, "set xlabel 'k (итерации)'\n");
fprintf(gnuplot, "set ylabel 'x (значение)'\n");
fprintf(gnuplot, "set grid\n");
fprintf(gnuplot, "set key outside\n");

// Построение графика для всех столбцов
fprintf(gnuplot, "plot 'data2.txt' using 0:1 with lines title 'x0', \\\n");
fprintf(gnuplot, "'data2.txt' using 0:2 with lines title 'x1', \\\n");
fprintf(gnuplot, "'data2.txt' using 0:3 with lines title 'x2', \\\n");
fprintf(gnuplot, "'data2.txt' using 0:4 with lines title 'x3', \\\n");
fprintf(gnuplot, "'data2.txt' using 0:5 with lines title 'x4'\n");

fflush(gnuplot);  // Очищаем буфер gnuplot
pclose(gnuplot);  // Закрываем gnuplot
```

Рис 3.7 Построение графика на языке Си

График выглядит так:

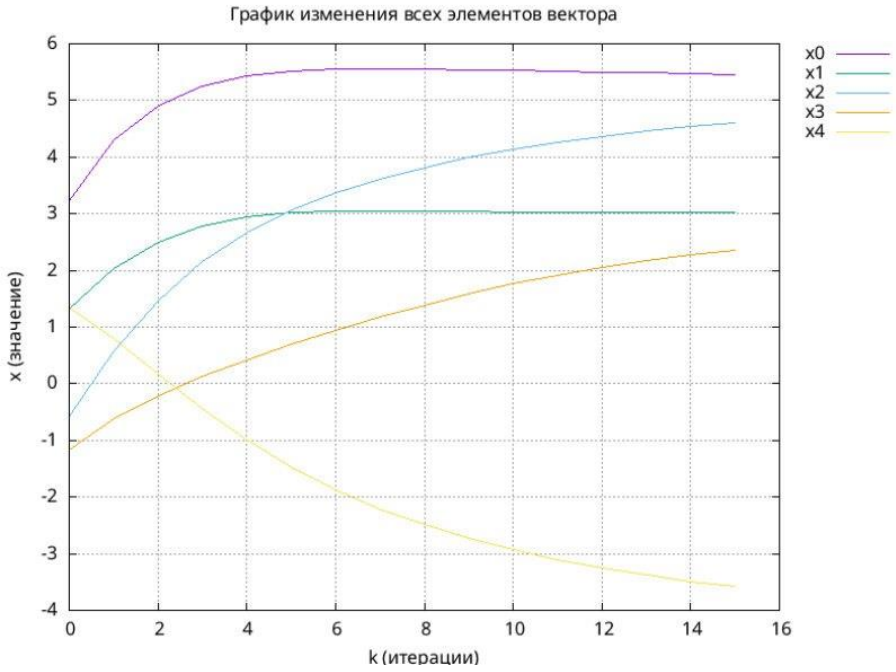


Рис 3.8 График изменения элементов X

3.4 Выводы по работе

На основании анализа графика зависимости количества итераций от выбранного значения точности можно сделать вывод о том, что для достижения более высокой точности вычислений необходимо увеличить количество итераций.

Сопоставив результаты работы обоих методов, можно отметить, что метод Жордана-Гаусса дал практически идентичные ответы, полученные в результате применения метода Гаусса-Зейделя с точностью 0.001. При этом метод Гаусса-Зейделя демонстрирует более высокую скорость работы, однако требует проверки условий сходимости.

4. Лабораторная работа 4. Интерполирование экспериментальных зависимостей.

4.1. Цель работы. Задача. Вариант задания

Цель работы – закрепление знаний об интерполировании экспериментальных зависимостей; получение навыков численного расчета интерполирующих зависимостей с использованием различных методов.

Задача - Получить интерполирующие зависимости для полиномов различных степеней по формулам Лагранжа и Ньютона с использованием табличного процессора MS Excel. Оформить результаты расчетов в табличной и графической формах. Провести исследование и сделать выводы о влиянии различных факторов на точность описания экспериментальных данных методом интерполирования

Номер варианта: 66

№	x	y(x)
1	-2,50	-13,29
2	-2,20	-13,50
3	-1,90	-13,39
4	-1,60	-13,10
5	-1,30	-12,70
6	-1,00	-12,02
7	-0,70	-11,23
8	-0,40	-10,23
9	-0,10	-9,02
10	0,20	-7,73
11	0,50	-6,33
12	0,80	-4,74
13	1,10	-2,97

Номера точек для расчёта полинома 2-й степени по формуле Лагранжа:
1 6 11

Номера точек для расчёта полинома 3-й степени по формуле Ньютона:
1 4 9 12

4.2. Теоретическая часть

Задача интерполирования экспериментальных данных ставится следующим образом (рис. 6.1). Имеется функциональная зависимость вида *y = F(x)*, 48 полученная в ходе экспериментальных наблюдений и заданная в табличной форме *n + 1* парами значений, называемыми узлами интерполяции (Mi).

При решении задачи интерполирования требуется найти интерполирующую функцию известного класса *y = f(x)*, принимающую в узлах интерполяции те же значения, что были получены в ходе экспериментальных наблюдений. Геометрически это означает, что требуется найти определенного типа кривую *y = f(x)*, проходящую через заданную систему узловых точек, принадлежащих множеству *{M0, ..., Mn}*.

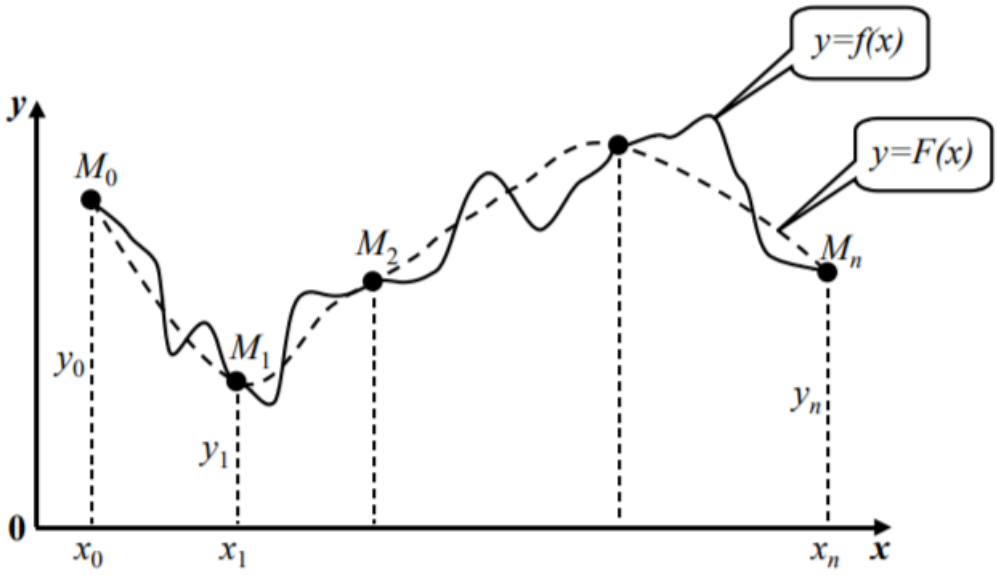


Рис. 6.1. Графическая иллюстрация задачи интерполирования

В данной постановке задача может иметь бесчисленное множество решений. Однако она становится однозначно определенной, если искать вместо произвольной функции полином степени m. В общем виде полиномиальная функция записывается как:

$$y = a_0 + a_1 x + a_2 x^2 + \ldots + a_m x^m$$

или свернуто:

$$y = \sum_{j=0}^{m} a_j x^j .$$

Частными случаями решения такой задачи являются функция вида y = const (полином нулевой степени), который можно получить, зная лишь один узел интерполяции, или линейная функция *y = a0 + a1x* (полином 1-й степени), получающийся из двух узловых точек. В общем случае для получения интерполяционного полинома степени *m* необходимо выбрать *m + 1* узел интерполяции из имеющегося множества *{M0, ..., Mn}*.

Таким образом, задача интерполирования сводится к получению достаточно простой интерполяционной функции *y = f(x)*, совпадающей в использованных для этого узлах интерполяции с известными экспериментальными значениями, а в остальных точках приближенно описывающей неизвестную закономерность с удовлетворительной точностью.

Принято говорить собственно об интерполяции, когда ведется поиск значений функции, описывающей данные, в точках, находящихся внутри экспериментального интервала изменения независимой переменной *(x0, xn)* и не являющихся узлами интерполяции. В то же время, говорят об экстраполяции, если ведется поиск аналогичных значений за пределами отрезка *[x0, xn]*.

Подбор степени интерполяционного полинома можно осуществить на основе информации о конечных разностях различных порядков.

Пусть *y = F(x)* – функция, заданная в виде множества полученных экспериментально узлов интерполяции, причем все ее значения (yi) получены в равноотстоящих друг от друга точках

(xi): xi+1 – xi = Δx = const, ∀i = 0, n –1.

Тогда выражения *Δyi = yi+1 – yi , ∀i = 0, n –1* называются конечными разностями первого порядка. Очевидно, если узлов интерполяции *n +1* , то количество конечных разностей первого порядка n . На основе информации о конечных разностях более низкого порядка определяются конечные разности более высоких порядков (m):

$$\Delta^m y_i = \Delta^{m-1} y_{i+1} - \Delta^{m-1} y_i, \forall i = \overline{0, n-m}.$$

Таким образом, при *n + 1* узлах интерполяции максимально возможная величина порядка конечной разности равна n, однако для определения наиболее подходящей степени интерполяционного полинома достаточно рассчитать и проанализировать конечные разности вплоть до порядка *n – 1*.

Конечные разности различных порядков удобно записывать в виде диагональной таблицы конечных разностей (табл. 6.1).

Конечные разности позволяют составить впечатление о важной для интерполяции «степени гладкости» рассматриваемой функции *y = F(x)*.

Выбор степени интерполяционного полинома основывается на следую-

50 щем правиле: она должна совпадать с порядком практически постоянных (в пределах точности полученных экспериментальных значений) конечных разностей. Для использования этого правила достаточно оценить величину разности между максимальным и минимальным значениями конечных разностей каждого порядка:

$$d_j = \max_i \{\Delta^j y_i\} - \min_i \{\Delta^j y_i\}.$$

Таблица 6.1

Диагональная таблица конечных разностей

x	y	Δy	$\Delta^2 y$	...	$\Delta^j y$	...	$\Delta^{n-1} y$
x_0	y_0	Δy_0	$\Delta^2 y_0$	…	$\Delta^j y_0$	…	$\Delta^{n-1} y_0$
x_1	y_1	Δy_1	$\Delta^2 y_1$	…	$\Delta^j y_1$	…	$\Delta^{n-1} y_1$
x_2	y_2	Δy_2	$\Delta^2 y_2$	…	…	…	
x_3	y_3	Δy_3	$\Delta^2 y_3$	…	$\Delta^j y_i$	…	
…	…	…	…	…	…	…	
x_i	y_i	Δy_i	$\Delta^2 y_i$	…	$\Delta^j y_{n-j}$		
x_{i+1}	y_{i+1}	Δy_{i+1}	…	…			
…	…	…	Δy_{n-2}				
x_{n-1}	y_{n-1}	Δy_{n-1}					
x_n	y_n						
		d_1	d_2	…	d_j	…	d_{n-1}

Порядок m с наименьшим разбросом dm в значениях конечных разностей указывает на искомую степень интерполяционного полинома.

Качество решения задачи интерполирования можно оценить по величине среднеквадратичной ошибки, рассчитанной во всех известных узлах интерполяции или тех узлах, которые не использовались для получения интерполяционного полинома:

$$E = \sqrt{\frac{1}{n+1}\sum_{i=0}^{n}(y_i - f(x_i))^2}\ .$$

Расчет интерполяционного полинома может быть выполнен с использованием формул Лагранжа или Ньютона.

Интерполяционный полином Лагранжа степени m записывается в виде:

$$f_m(x) = \sum_{i=0}^{m} y_i \frac{(x-x_0)\cdot(x-x_1)\cdot\ldots\cdot(x-x_{i-1})\cdot(x-x_{i+1})\cdot\ldots\cdot(x-x_m)}{(x_i-x_0)\cdot(x_i-x_1)\cdot\ldots\cdot(x_i-x_{i-1})\cdot(x_i-x_{i+1})\cdot\ldots\cdot(x_i-x_m)}$$

или свернуто:

$$f_m(x)=\sum_{i=0}^{m} y_i \frac{\prod_{\substack{j=0 \\ j\neq i}}^{m}(x-x_j)}{\prod_{j=0}^{m}(x_i-x_j)}.$$

Для расчета интерполяционного полинома степени m по формуле Ньютона требуется найти значения разделенных разностей вплоть до m-го порядка, для чего требуется m + 1 узел интерполяции.

Разделенная разность первого порядка определяется следующим образом:

$$\delta_i = \frac{y_{i+1}-y_i}{x_{i+1}-x_i}, \quad \forall i = \overline{0, n-1}.$$

Разделенные разности более высоких порядков определяются на основе информации о разделенных разностях более низких порядков:

$$\delta_i^m = \frac{\delta_{i+1}^{m-1}-\delta_i^{m-1}}{x_{i+m}-x_i}, \quad \forall i = \overline{0, n-m}.$$

Разделенные разности различных порядков удобно записывать в виде диагональной таблицы разделенных разностей (табл. 6.2).

Диагональная таблица разделенных разностей

x	y	δy	$\delta^2 y$	...	$\delta^j y$	...	$\delta^m y$
x_0	y_0	δy_0	$\delta^2 y_0$	…	$\delta^j y_0$	…	$\Delta^m y_0$
x_1	y_1	δy_1	$\delta^2 y_1$	…	…	…	
x_2	y_2	δy_2	…	…	$\delta^j y_i$	…	
…	…	…	$\delta^2 y_i$	…	…	…	
x_i	y_i	δy_i	…	…	$\delta^j y_{m-i}$		
x_{i+1}	y_{i+1}	…	$\delta^2 y_{m-2}$				
…	…	δy_{m-1}					
x_m	y_m						

Интерполяционный полином Ньютона записывается в виде:

$$f_m(x)=y_0+\delta y_0(x-x_0)+\delta^2 y_0(x-x_0)(x-x_1)+\ldots+\delta^m y_0(x-x_0)(x-x_1)\cdot\ldots\cdot(x-x_{m-1})$$

или свернуто:

$$f_m(x)=y_0+\sum_{j=1}^{m}\left[\delta^j y_0\cdot\prod_{i=0}^{j-1}(x-x_i)\right].$$

На самом деле интерполяционные формулы Лагранжа и Ньютона представляют собой различные формы записи одной и той же зависимости. Однако формула Ньютона удобна тем, что при добавлении нового узла интерполяции все ранее найденные члены остаются без изменения и в полиноме добавляется лишь один дополнительный член. Это позволяет,

последовательно добавляя по одному дополнительному узлу, при необходимости, постепенно наращивать сложность интерполирующей зависимости.

4.3. Практическая часть

Выполним лабораторную работу с использованием языка программирования C++. Для начала данные считываются из файла, а затем выводятся для наглядности в файл output_data.txt

output_data.txt

	x	y
2	-2.5	-13.29
3	-2.2	-13.5
4	-1.9	-13.39
5	-1.6	-13.1
6	-1.3	-12.7
7	-1	-12.02
8	-0.7	-11.23
9	-0.4	-10.23
10	-0.1	-9.02
11	0.2	-7.73
12	0.5	-6.33
13	0.8	-4.74
14	1.1	-2.97

Рис. 4.1 Вывод экспериментальных данных

Далее строится график, который отображает положение точек экспериментальных данных на координатной плоскости.

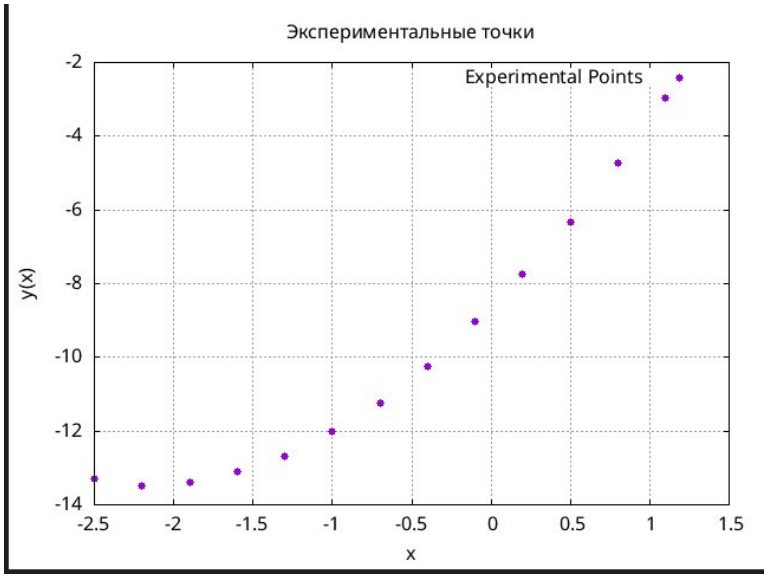


Рис. 4.2 График экспериментальных данных

Построение графика реализовано с помощью библиотеки gnuplot

```cpp
void plotData(const vector<Point> &data) {
    ofstream dataFile("output_data.txt");
    ofstream gnuplotFile("plot_commands.gp");

    if (dataFile.is_open()) {
        dataFile << "  x     y" << endl;
        for (const auto &point : data) {
            dataFile << point.x << " " << point.y << endl;
        }
        dataFile.close();
    } else {
        cerr << "Ошибка: не удалось записать данные в файл output_data.txt" << endl;
        return;
    }

    if (gnuplotFile.is_open()) {
        gnuplotFile << "set title 'Экспериментальные точки'\n";
        gnuplotFile << "set xlabel 'x'\n";
        gnuplotFile << "set ylabel 'y(x)'\n";
        gnuplotFile << "set grid\n";
        gnuplotFile << "set term png\n";
        gnuplotFile << "set output 'output_plot.png'\n";
        gnuplotFile << "plot 'output_data.txt' using 1:2 with points pointtype 7 title 'Experimental Points'\n";
        gnuplotFile.close();

        if (system("gnuplot plot_commands.gp") == -1) {
            cerr << "Ошибка: не удалось запустить Gnuplot" << endl;
        } else {
            cout << "График успешно сохранен в output_plot.png" << endl;
        }
    } else {
        cerr << "Ошибка: не удалось создать файл команд для Gnuplot" << endl;
    }
}
```

Рис. 4.3 Реализация графика на C++

После построения графика строится таблица конечных разностей

finite_differences.txt

```
Таблица конечных разностей

    y(x)      Δ^1y      Δ^2y      Δ^3y      Δ^4y      Δ^5y      Δ^6y      Δ^7y      Δ^8y      Δ^9y      Δ^10y      Δ^11y      Δ^12y
  -13.29     -0.21      0.32     -0.14      0.07      0.17     -0.75      1.94     -4.11      7.6     -12.43      17.9    -22.22
   -13.5      0.11      0.18     -0.07      0.24     -0.58      1.19     -2.17      3.49     -4.83      5.47     -4.32
  -13.39      0.29      0.11      0.17     -0.34      0.61     -0.98      1.32     -1.34      0.64      1.15
   -13.1       0.4      0.28     -0.17      0.27     -0.37      0.34     -0.02      -0.7      1.79
   -12.7      0.68      0.11       0.1      -0.1     -0.03      0.32     -0.72      1.09
  -12.02      0.79      0.21         0     -0.13      0.29      -0.4      0.37
  -11.23         1      0.21     -0.13      0.16     -0.11     -0.03
  -10.23      1.21      0.08      0.03      0.05     -0.14
   -9.02      1.29      0.11      0.08     -0.09
   -7.73       1.4      0.19     -0.01
   -6.33      1.59      0.18
   -4.74      1.77
   -2.97

Разницы между максимальной и минимальной конечной разностью для каждого столбца:
Для столбца Δ^1y: 1.98
Для столбца Δ^2y: 0.24
Для столбца Δ^3y: 0.34
Для столбца Δ^4y: 0.61
Для столбца Δ^5y: 1.19
Для столбца Δ^6y: 2.17
Для столбца Δ^7y: 4.11
Для столбца Δ^8y: 7.6
Для столбца Δ^9y: 12.43
Для столбца Δ^10y: 17.9
Для столбца Δ^11y: 22.22

Оптимальная степень интерполяционного полинома
Укажите нужную степень на основе значений конечных разностей.
Минимальная разница: 0.24
Номер полинома: 2
```

Рис. 4.4 Таблица конечных разностей

Используя полином Лагранжа 2й степени выводим результаты интерполирования данных.

```cpp
vector<Point> lagrangeInterpolation(const std::vector<Point>& points, const vector<int> lagPoints) {
    vector<Point> result;

    for (const auto& point : points) {
        double lagrangeY = 0;

        for (int i = 0; i < 3; ++i) {
            double Li = 1.0;

            for (int j = 0; j < 3; ++j) {
                if (i != j) {
                    Li *= (point.x - points[lagPoints[j]-1].x) /
                          (points[lagPoints[i]-1].x - points[lagPoints[j]-1].x);
                }
            }

            lagrangeY += Li * points[lagPoints[i]-1].y;
        }

        result.push_back({point.x, lagrangeY});
    }

    return result;
}
```

Рис. 4.5 Интерполирование полиномом Лагранжа 2й степени

```cpp
vector<Point> newtonInterpolation(const vector<Point>& data, const vector<int>& newtonPoints) {
    vector<Point> result;
    int m = newtonPoints.size();

    if (m != 4) {
        cerr << "Ошибка: массив newtonPoints должен содержать ровно 4 индекса." << endl;
        return result;
    }

    vector<vector<double>> differences(m, vector<double>(m, 0.0));
    for (int i = 0; i < m; i++) {
        differences[i][0] = data[newtonPoints[i]-1].y;
    }
    for (int j = 1; j < m; j++) {
        for (int i = 0; i < m - j; i++) {
            differences[i][j] = (differences[i + 1][j - 1] - differences[i][j - 1]) /
                                (data[newtonPoints[i + j]-1].x - data[newtonPoints[i]-1].x);
        }
    }

    for (const auto& point : data) {
        double newtonY = differences[0][0];
        double product = 1.0;

        for (int j = 1; j < m; j++) {
            product *= (point.x - data[newtonPoints[j - 1]-1].x);
            newtonY += differences[0][j] * product;
        }

        result.push_back({point.x, newtonY});
    }

    return result;
}
```

Рис. 4.6 Интерполирование полиномом Ньютона 3й степени

Полученные результаты интерполирования выводятся в файлы lagrangeAnswer.txt и newtonAnswer.txt

lagrangeAnswer.txt

	x	Fn(x)
1		
2	-2.50	-13.29
3	-2.20	-13.39
4	-1.90	-13.31
5	-1.60	-13.06
6	-1.30	-12.63
7	-1.00	-12.02
8	-0.70	-11.24
9	-0.40	-10.27
10	-0.10	-9.14
11	0.20	-7.82
12	0.50	-6.33
13	0.80	-4.66
14	1.10	-2.82

Рис. 4.7 Результаты интерполирования данных полиномом Лагранжа

newtonAnswer.txt

	x	Fn(x)
1		
2	-2.50	-13.29
3	-2.20	-13.44
4	-1.90	-13.37
5	-1.60	-13.10
6	-1.30	-12.63
7	-1.00	-11.98
8	-0.70	-11.16
9	-0.40	-10.17
10	-0.10	-9.02
11	0.20	-7.73
12	0.50	-6.30
13	0.80	-4.74
14	1.10	-3.07

Рис. 4.8 Результаты интерполирования данных полиномом Ньютона

По полученным данным строим графики зависимостей

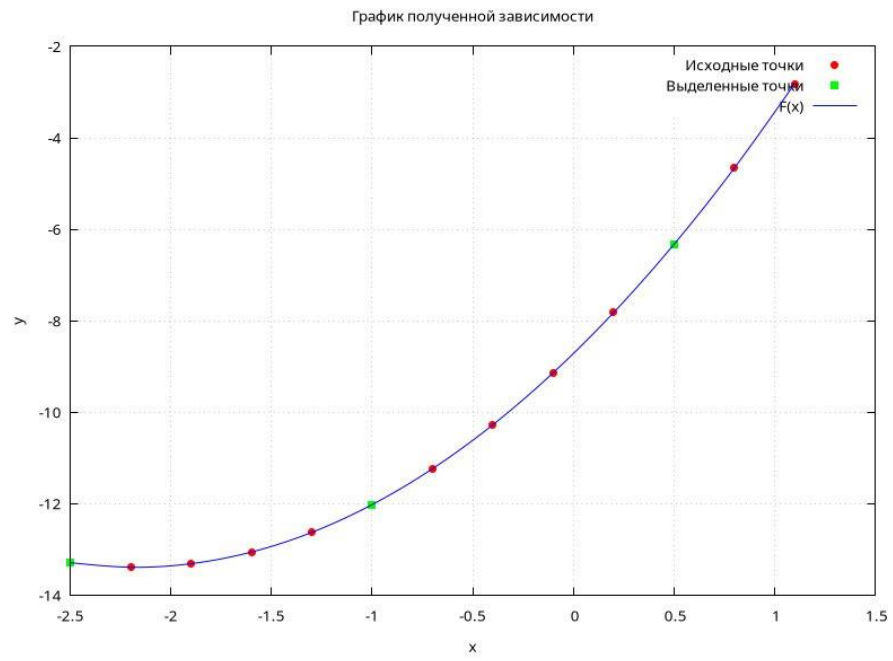


Рис. 4.9 График полученной зависимости (1)

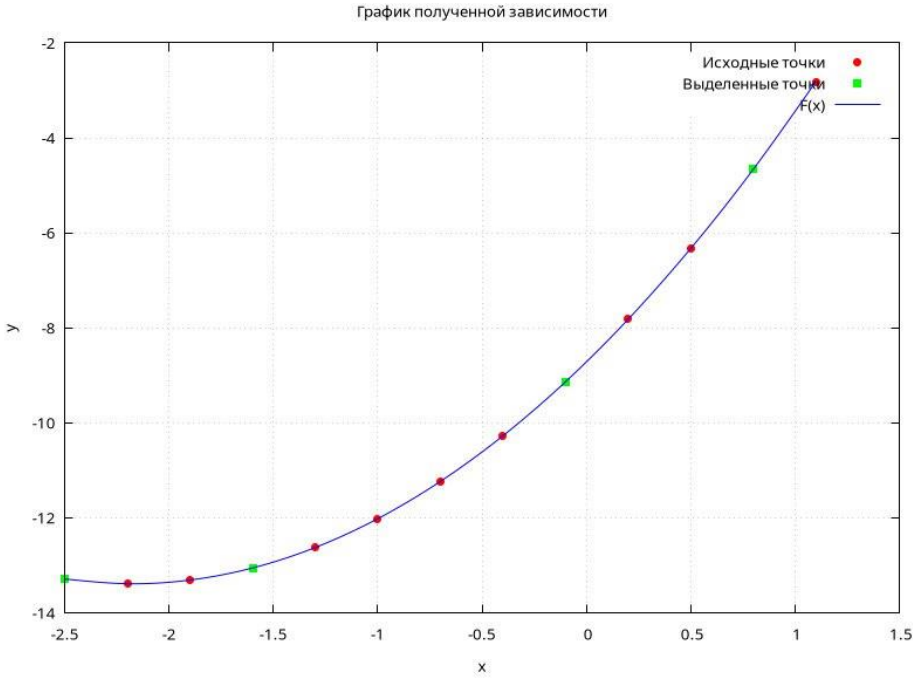


Рис. 4.10 График полученной зависимости (2)

Далее считаются квадраты разностей между полученными значениями функции и экспериментальными данными

mistakes.txt

	Лагранж	Ньютон
2	0.0000	0.0000
3	0.0122	0.0039
4	0.0060	0.0004
5	0.0017	0.0000
6	0.0052	0.0043
7	0.0000	0.0013
8	0.0000	0.0051
9	0.0020	0.0039
10	0.0135	0.0000
11	0.0084	0.0000
12	0.0000	0.0011
13	0.0061	0.0000
14	0.0236	0.0092

Рис. 4.11 Значения ошибок

Из полученных данных находим среднеквадратичную ошибку во всех точках и во всех, кроме использованных в получении интерполяционного полинома

```cpp
vector<double> quadr_mistake(const vector<Point> &mistakes, const vector<int> &lp, const vector<int> &np){
    double sumX = 0.0, sumY = 0.0;
    for (const auto& point : mistakes) {
        sumX += point.x;
        sumY += point.y;
    }

    double meanX = sumX / mistakes.size();
    double meanY = sumY / mistakes.size();

    std::unordered_set<int> lpSet(lp.begin(), lp.end());
    std::unordered_set<int> npSet(np.begin(), np.end());

    double filteredSumX = 0.0, filteredSumY = 0.0;
    int filteredCountX = 0, filteredCountY = 0;

    for (size_t i = 0; i < mistakes.size(); ++i) {
        if (!lpSet.count(i)) {
            filteredSumX += mistakes[i].x;
            ++filteredCountX;
        }
        if (!npSet.count(i)) {
            filteredSumY += mistakes[i].y;
            ++filteredCountY;
        }
    }

    double filteredMeanX = (filteredCountX > 0) ? (filteredSumX / filteredCountX) : 0.0;
    double filteredMeanY = (filteredCountY > 0) ? (filteredSumY / filteredCountY) : 0.0;

    return {std::sqrt(meanX),
            std::sqrt(meanY),
            std::sqrt(filteredMeanX),
            std::sqrt(filteredMeanY)};
}
```

Рис. 4.12 Нахождение среднеквадратичной ошибки

Полученные результаты среднеквадратичных ошибок выводятся в файл quadr_mistakes.txt

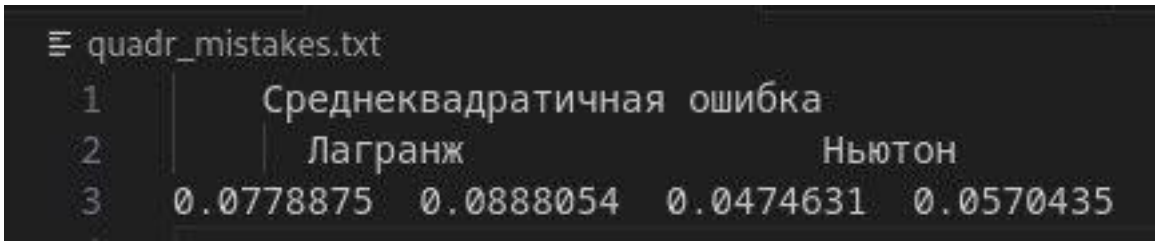


Рис. 4.13 Среднеквадратичные ошибки

4.4 Выводы по работе